

BÀI THỰC HÀNH · 90-120 PHÚT

MINI LAB

DATABASE TESTING & DATA GENERATION

Môn học

Kiểm thử Phần mềm / Software Quality Assurance

Chủ đề

Database Testing & Test Data Generation cho Hệ thống E-Commerce

Công nghệ

Node.js · SQLite · Jest · Supertest · @faker-js/faker

Thư viện DB

Knex / Better-SQLite3 / sqlite3

Sản phẩm nộp

Test data script · Test suite · Bug report

Mục lục

1	Tổng quan và mục tiêu	1
1.1	Kết quả học tập mong đợi	1
1.2	Các khái niệm then chốt	1
2	Bối cảnh hệ thống và cấu trúc cơ sở dữ liệu	1
2.1	Điểm cần lưu ý khi kiểm thử	2
2.2	Sơ đồ quan hệ tổng quát	3
3	Yêu cầu chi tiết	3
3.1	Phần 1 — Test Data Generation & Management	3
3.2	Phần 2 — Database Integrity Testing	5
3.3	Phần 3 — Database Unit / Integration Testing	6
3.4	Phần 4 — Báo cáo kết quả kiểm thử	8
4	Starter kit	9
5	Quy trình thực hiện đề xuất	11
6	Tiêu chí đánh giá tham khảo	11
7	Sản phẩm cần nộp	11

1. Tổng quan và mục tiêu

Trong bài lab này, sinh viên đóng vai trò **QA Engineer**, thực hiện kiểm thử mức tích hợp (Integration Testing) và kiểm thử cơ sở dữ liệu (Database Testing) cho một phân hệ E-Commerce, bao gồm các chức năng thuộc **Pool B: Cart & Checkout** và **Pool C: Web Admin**.

Vì sao bài lab này quan trọng?

Trong thực tế, nhiều lỗi nghiêm trọng của hệ thống thương mại điện tử không nằm ở giao diện mà ở tầng logic nghiệp vụ tương tác với cơ sở dữ liệu: sai công thức tính tiền, thiếu ràng buộc khóa ngoại, sai luồng trạng thái đơn hàng. Các lỗi này thường chỉ lộ ra khi dữ liệu chạm giá trị biên hoặc xuất hiện bất thường, nên rất khó phát hiện nếu chỉ kiểm thử thủ công theo luồng thông thường.

1.1. Kết quả học tập mong đợi

Sau bài lab, sinh viên có thể:

- **Test Data Generation & Management:** thiết kế script tự động sinh dữ liệu test bao phủ các kịch bản biên.
- **Database Integrity Testing:** kiểm tra tính toàn vẹn dữ liệu, constraints và data anomalies.
- **Database Integration Testing:** dùng Jest/Supertest để kiểm thử API tương tác trực tiếp với database.
- **Bug Reporting:** phát hiện hidden bugs, xác định nguyên nhân gốc và đề xuất bản vá.

1.2. Các khái niệm then chốt

Khái niệm	Ý nghĩa
Integration Testing	Kiểm thử sự phối hợp giữa các thành phần (ở đây là API ↔ Database), thay vì từng hàm riêng lẻ như Unit Test.
Database Testing	Kiểm thử tính đúng đắn, toàn vẹn và nhất quán của dữ liệu lưu trữ, không chỉ kiểm thử code xử lý.
Boundary Testing	Tập trung vào các giá trị biên: min/max, hết hạn/còn hạn, đủ/thiếu điều kiện—nơi lỗi thường xuất hiện nhất.
Data Anomaly	Dữ liệu bất thường do thiếu ràng buộc DB; ví dụ bản ghi con còn tồn tại sau khi bản ghi cha đã bị xóa.
Hidden Bug	Lỗi không lộ ra trong happy path mà chỉ xuất hiện khi test trường hợp biên hoặc dữ liệu sai lệch.

2. Bối cảnh hệ thống và cấu trúc cơ sở dữ liệu

Hệ thống E-Commerce sử dụng SQLite. Lược đồ sơ khởi gồm năm bảng sau.

Database schema ban đầu

```
-- 1. Bảng Người dùng
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL
);

-- 2. Bảng Sản phẩm
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  price REAL NOT NULL,
  stock INTEGER DEFAULT 0
);

-- 3. Bảng Mã giảm giá
CREATE TABLE coupons (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  code TEXT UNIQUE NOT NULL,
  discount_type TEXT CHECK(discount_type IN ('percent', 'fixed')) NOT NULL,
  discount_value REAL NOT NULL,
  min_order_amount REAL DEFAULT 0,
  max_uses_per_user INTEGER DEFAULT 1,
  expired_at DATETIME NOT NULL,
  is_active INTEGER DEFAULT 1
);

-- 4. Bảng Lịch sử sử dụng Coupon
CREATE TABLE coupon_usage (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  coupon_id INTEGER,
  user_id INTEGER,
  used_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 5. Bảng Đơn hàng
CREATE TABLE orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  total_amount REAL NOT NULL,
  discount_amount REAL DEFAULT 0,
  final_amount REAL NOT NULL,
  status TEXT CHECK(status IN
    ('pending', 'confirmed', 'shipping', 'delivered', 'canceled'))
    DEFAULT 'pending'
);
```

2.1. Điểm cần lưu ý khi kiểm thử

- **users:** ràng buộc email UNIQUE NOT NULL; cần test insert trùng email.
- **products:** trường price REAL; cần xác minh kiểu dữ liệu qua API luôn nhất quán.
- **coupons:** CHECK(discount_type IN (...)) là domain constraint cần được khai thác khi test.
- **orders:** status mô phỏng state machine gồm năm trạng thái cố định.

Về logic, bảng này cần khóa ngoại tới `coupons(id)` và `users(id)`, nhưng starter kit cố tình bỏ sót. Sinh viên cần chứng minh hậu quả: khi bản ghi cha bị xóa, dữ liệu rác (orphan record) vẫn tồn tại.

2.2. Sơ đồ quan hệ tổng quát



3. Yêu cầu chi tiết

3.1. Phần 1 — Test Data Generation & Management

Tạo file `generate-data.js` chứa các hàm sinh dữ liệu tự động phục vụ kiểm thử.

Mục đích

Không chỉ dùng dữ liệu “đẹp”. Cần chủ động tạo dữ liệu đại diện cho từng trường hợp biên để test độc lập, có dữ liệu sẵn sàng và có thể tái lập ở mọi lần chạy.

3.1.1. Sinh dữ liệu biên cho coupon

Mã coupon	Mục đích	Thiết lập dữ liệu
CP_EXPIRED	Coupon hết hạn	<code>expired_at</code> nhỏ hơn ngày hiện tại; hệ thống phải từ chối.
CP_INACTIVE	Coupon bị khóa	Còn hạn nhưng <code>is_active = 0</code> ; vẫn phải bị từ chối.
CP_PERCENT	Giảm theo phần trăm	Giảm 10%, áp dụng cho đơn từ 100; dùng kiểm tra công thức.
CP_FIXED	Giảm cố định	Giảm 15 đơn vị tiền tệ, không phụ thuộc tổng đơn hàng.
CP_MAX_REACHED	Giới hạn lượt dùng	User 1 đã đạt tối đa; chèn sẵn lịch sử vào <code>coupon_usage</code> .

3.1.2. Quản lý vòng đời test: Setup / Teardown

Xây dựng hai hàm:

- `setupTestDB()`: làm sạch bảng và insert bộ dữ liệu chuẩn trước mỗi test suite/test case.
- `teardownTestDB()`: đóng kết nối và dọn tài nguyên sau khi test kết thúc.

Nguyên tắc Test Isolation

Mỗi test không được phụ thuộc vào dữ liệu còn lại từ test trước. Setup/teardown không đúng có thể làm test pass hoặc fail ngẫu nhiên theo thứ tự chạy—một dạng **flaky test**.

Gợi ý khung generate-data.js

```
const { faker } = require('@faker-js/faker');

function setupTestDB(db) {
  return new Promise((resolve, reject) => {
    db.serialize(() => {
      db.run(`DELETE FROM coupon_usage`);
      db.run(`DELETE FROM coupons`);
      db.run(`DELETE FROM orders`);
      db.run(`DELETE FROM products`);
      db.run(`DELETE FROM users`);

      db.run(`INSERT INTO users (id, email, name)
              VALUES (1, 'test@example.com', 'Test User')`);

      db.run(`INSERT INTO coupons
              (code, discount_type, discount_value, min_order_amount,
               max_uses_per_user, expired_at, is_active)
              VALUES ('CP_EXPIRED', 'fixed', 10, 0, 1, '2020-01-01', 1)`);

      db.run(`INSERT INTO coupons
              (code, discount_type, discount_value, min_order_amount,
               max_uses_per_user, expired_at, is_active)
              VALUES ('CP_INACTIVE', 'fixed', 10, 0, 1, '2030-01-01', 0)`);

      db.run(`INSERT INTO coupons
              (code, discount_type, discount_value, min_order_amount,
               max_uses_per_user, expired_at, is_active)
              VALUES ('CP_PERCENT', 'percent', 0.10, 100, 5, '2030-01-01', 1)`);

      db.run(`INSERT INTO coupons
              (code, discount_type, discount_value, min_order_amount,
               max_uses_per_user, expired_at, is_active)
              VALUES ('CP_FIXED', 'fixed', 15, 0, 5, '2030-01-01', 1)`);

      db.run(`INSERT INTO coupons
              (id, code, discount_type, discount_value, min_order_amount,
               max_uses_per_user, expired_at, is_active)
              VALUES (5, 'CP_MAX_REACHED', 'fixed', 10, 0, 1, '2030-01-01', 1)`);

      db.run(`INSERT INTO coupon_usage (coupon_id, user_id)
              VALUES (5, 1)`, (err) => err ? reject(err) : resolve());
    });
  });
}
```

```
function teardownTestDB(db) {
  return new Promise((resolve, reject) => {
    db.close((err) => err ? reject(err) : resolve());
  });
}

module.exports = { setupTestDB, teardownTestDB };
```

Lưu ý

Đây là sườn tham khảo. Sinh viên phải tự điều chỉnh ID và bổ sung dữ liệu products, orders phù hợp với các test case ở Phần 3.

3.2. Phần 2 — Database Integrity Testing

Viết test trong db-logic.test.js để kiểm tra cấu trúc và tính toàn vẹn của DB.

3.2.1. Referential Integrity

1. Giả lập xóa user: `DELETE FROM users WHERE id = 1`.
2. Kiểm tra các bản ghi liên quan trong orders hoặc coupon_usage.
3. Kết quả đúng phải là: bản ghi con được xóa theo cascade, hoặc DB chặn lệnh xóa.

Test phát hiện orphan record

```
test('Referential Integrity: orphan record sau khi xóa user', (done) => {
  db.run(`DELETE FROM users WHERE id = 1`, () => {
    db.all(`SELECT * FROM coupon_usage WHERE user_id = 1`, (err, rows) => {
      // Nếu FK + ON DELETE CASCADE đúng, rows phải rỗng.
      expect(rows.length).toBe(0); // Dự kiến FAIL: chứng minh lỗi hỏng.
      done();
    });
  });
});
```

3.2.2. Constraints và Data Anomaly

- **Unique Constraint:** insert hai coupon trùng code; xác minh lỗi `SQLITE_CONSTRAINT`.
- **Data Type Consistency:** gọi GET `/api/products/:id` với nhiều ID; price luôn phải là number.

Test unique constraint và kiểu dữ liệu

```
test('Unique Constraint: không cho phép trùng code coupon', (done) => {
  db.run(`INSERT INTO coupons
    (code, discount_type, discount_value, expired_at)
  VALUES ('CP_FIXED', 'fixed', 5, '2030-01-01')`, (err) => {
    expect(err).not.toBeNull();
    expect(err.code).toBe('SQLITE_CONSTRAINT');
    done();
  });
});
```

```
});

test('Data Type Consistency: price luôn phải là number', async () => {
  for (let id = 1; id <= 5; id++) {
    const res = await request(app).get(`/api/products/${id}`);
    if (res.statusCode === 200) {
      expect(typeof res.body.price).toBe('number');
    }
  }
});
```

Starter kit cố tình ép price thành string khi ID sản phẩm là số chẵn. Test lặp qua nhiều ID sẽ phát hiện API không bảo đảm data contract nhất quán.

3.3. Phần 3 — Database Unit / Integration Testing

Sử dụng Jest và Supertest để kiểm thử các API nghiệp vụ tương tác với database.

3.3.1. API POST /api/apply-coupon — FR09

Case	Dữ liệu / hành vi	Kết quả mong đợi	Mục tiêu
Case 1 · Valid	Mã hợp lệ, đơn 200	Giảm đúng; final amount chính xác	Happy path và công thức
Case 2 · Below Min	Tổng đơn nhỏ hơn mức tối thiểu	400 Bad Request	Ràng buộc điều kiện
Case 3 · Expired	Dùng CP_EXPIRED	400 Bad Request	Boundary theo thời gian
Case 4 · Max Usage	User đã dùng hết lượt	400 Bad Request	Giới hạn theo user

Code nguồn dùng $\text{total_amount} * (1 - \text{coupon.discount_value})$. Với đơn 200 và mức giảm 10%, giá trị đúng phải là 20, final amount là 180. Assertion phải dùng số liệu tính tay để làm lộ lỗi.

Bốn test case cho apply-coupon

```
test('Case 1: CP_PERCENT giảm đúng 10% cho đơn 200', async () => {
  const res = await request(app).post('/api/apply-coupon').send({
    user_id: 1, coupon_code: 'CP_PERCENT', total_amount: 200
  });
  expect(res.statusCode).toBe(200);
  expect(res.body.discount_amount).toBe(20);
  expect(res.body.final_amount).toBe(180);
});
```



```

});

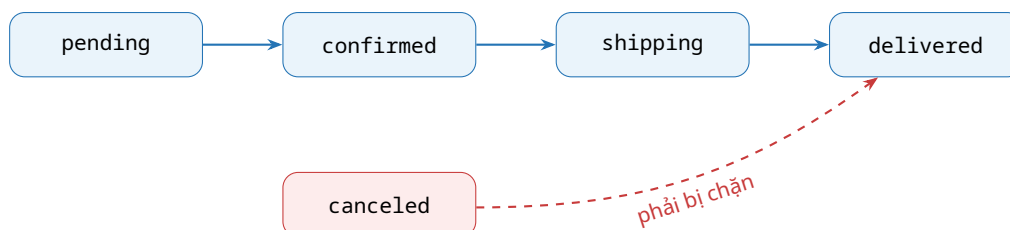
test('Case 2: đơn hàng chưa đạt min_order_amount', async () => {
  const res = await request(app).post('/api/apply-coupon').send({
    user_id: 1, coupon_code: 'CP_PERCENT', total_amount: 50
  });
  expect(res.statusCode).toBe(400);
});

test('Case 3: coupon đã hết hạn', async () => {
  const res = await request(app).post('/api/apply-coupon').send({
    user_id: 1, coupon_code: 'CP_EXPIRED', total_amount: 100
  });
  expect(res.statusCode).toBe(400);
});

test('Case 4: user đã dùng hết lượt', async () => {
  const res = await request(app).post('/api/apply-coupon').send({
    user_id: 1, coupon_code: 'CP_MAX_REACHED', total_amount: 100
  });
  expect(res.statusCode).toBe(400);
});

```

3.3.2. API PUT /api/admin/orders/:id/status — FR10 & FR18



- **Valid Transition:** pending → confirmed → shipping → delivered.
- **Invalid Transition Bug Hunter:** canceled → delivered phải trả về 400.

State Transition Testing

canceled là terminal state: không được chuyển sang trạng thái khác. Việc cho phép một đơn đã hủy thành đã giao sẽ làm sai dữ liệu thống kê và có thể ảnh hưởng thanh toán hoặc hoàn tiền.

Test state machine của đơn hàng

```

test('Valid: pending -> confirmed -> shipping -> delivered', async () => {
  let res = await request(app)
    .put('/api/admin/orders/1/status').send({ status: 'confirmed' });
  expect(res.statusCode).toBe(200);

  res = await request(app)
    .put('/api/admin/orders/1/status').send({ status: 'shipping' });
  expect(res.statusCode).toBe(200);

  res = await request(app)
    .put('/api/admin/orders/1/status').send({ status: 'delivered' });

```

```

    expect(res.statusCode).toBe(200);
  });

  test('Invalid: canceled -> delivered phải bị chặn', async () => {
    // Order #2 cần được setup sẵn ở trạng thái canceled.
    const res = await request(app)
      .put('/api/admin/orders/2/status').send({ status: 'delivered' });
    expect(res.statusCode).toBe(400);
  });

```

3.4. Phần 4 — Báo cáo kết quả kiểm thử

Tạo file REPORT.md, trình bày danh sách lỗi/lỗ hổng đã phát hiện, nguyên nhân kỹ thuật, bằng chứng test và code fix patch đề xuất.

Cấu trúc REPORT.md đề xuất

```

# BÁO CÁO KIỂM THỬ - MINI LAB DATABASE TESTING

## Tổng quan
- Tổng số test case: ...
- Số lượng Pass: ...
- Số lượng Fail (phát hiện bug): ...

## Danh sách Bug phát hiện

### BUG-01: Sai công thức tính giảm giá theo phần trăm
- **Mức độ:** Cao (High)
- **Vị trí:** app.js - POST /api/apply-coupon
- **Mô tả:** Công thức tính sai giá trị giảm.
- **Bằng chứng:** Case 1 - test CP_PERCENT.
- **Đề xuất sửa:**
  ```javascript
 discount_amount = total_amount * (coupon.discount_value / 100);
  ```

### BUG-02: Cho phép canceled -> delivered
- **Mức độ:** Cao (High)
- **Vị trí:** app.js - PUT /api/admin/orders/:id/status
- **Đề xuất:** Xóa nhánh cho phép chuyển trạng thái sai.

### BUG-03: Thiếu Foreign Key trong coupon_usage
- **Mức độ:** Trung bình (Medium)
- **Vị trí:** Database schema
- **Mô tả:** Phát sinh orphan record khi xóa user/coupon.
- **Đề xuất sửa:**
  ```sql
 FOREIGN KEY (coupon_id) REFERENCES coupons(id) ON DELETE CASCADE,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
  ```

### BUG-04: API trả sai kiểu price
- **Mức độ:** Trung bình (Medium)
- **Vị trí:** app.js - GET /api/products/:id
- **Đề xuất:** Không ép row.price sang string.

## Kết luận
(Đánh giá thiết kế DB/code và khuyến nghị cải tiến.)

```

4. Starter kit

Sao chép mã dưới đây vào app.js. Mã có chứa lỗi ẩn được đánh dấu bằng comment **LỖI**. **Không sửa trực tiếp**; hãy viết test để phát hiện rồi đề xuất sửa trong báo cáo.

app.js — mã nguồn ứng dụng có chủ đích chứa lỗi

```
const express = require('express');
const sqlite3 = require('sqlite3').verbose();
const app = express();

app.use(express.json());
const db = new sqlite3.Database(':memory:');

db.serialize(() => {
  db.run(`CREATE TABLE users
    (id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT UNIQUE, name TEXT)`);
  db.run(`CREATE TABLE products
    (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, price REAL, stock INTEGER)`);
  db.run(`CREATE TABLE coupons
    (id INTEGER PRIMARY KEY AUTOINCREMENT, code TEXT UNIQUE,
    discount_type TEXT, discount_value REAL, min_order_amount REAL,
    max_uses_per_user INTEGER, expired_at DATETIME, is_active INTEGER)`);

  // LỖI THIẾT KẾ: không khai báo FOREIGN KEY!
  db.run(`CREATE TABLE coupon_usage
    (id INTEGER PRIMARY KEY AUTOINCREMENT, coupon_id INTEGER,
    user_id INTEGER, used_at DATETIME)`);

  db.run(`CREATE TABLE orders
    (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER,
    total_amount REAL, discount_amount REAL, final_amount REAL, status TEXT)`);
});

// API 1: Lấy thông tin sản phẩm
app.get('/api/products/:id', (req, res) => {
  const id = req.params.id;
  db.get(`SELECT * FROM products WHERE id = ?`, [id], (err, row) => {
    if (err || !row) return res.status(404).json({ error: 'Product not found' });

    // Cố tình ép sai kiểu dữ liệu ở các ID chẵn
    if (row.id % 2 === 0) row.price = row.price.toString();
    res.json(row);
  });
});

// API 2: Áp dụng coupon
app.post('/api/apply-coupon', (req, res) => {
  const { user_id, coupon_code, total_amount } = req.body;

  db.get(`SELECT * FROM coupons WHERE code = ?`, [coupon_code],
  (err, coupon) => {
    if (!coupon) return res.status(400).json({ error: 'Mã không tồn tại' });
    if (!coupon.is_active)
      return res.status(400).json({ error: 'Mã đã bị khóa' });
    if (new Date(coupon.expired_at) < new Date())
      return res.status(400).json({ error: 'Mã đã hết hạn' });
    if (total_amount < coupon.min_order_amount)
      return res.status(400).json({
        error: 'Chưa đạt giá trị đơn hàng tối thiểu'
      });
  });
});
```

```

    });

    db.get(`SELECT COUNT(*) as count FROM coupon_usage
           WHERE coupon_id = ? AND user_id = ?`,
    [coupon.id, user_id], (err, result) => {
        if (result.count >= coupon.max_uses_per_user) {
            return res.status(400).json({ error: 'Bạn đã dùng hết lượt mã này' });
        }

        let discount_amount = 0;
        if (coupon.discount_type === 'percent') {
            // LỖI: công thức phần trăm sai
            discount_amount = total_amount * (1 - coupon.discount_value);
        } else {
            discount_amount = coupon.discount_value;
        }

        const final_amount = Math.max(0, total_amount - discount_amount);
        res.json({ total_amount, discount_amount, final_amount, coupon_code });
    });
});

// API 3: Cập nhật trạng thái đơn hàng (Admin)
app.put('/api/admin/orders/:id/status', (req, res) => {
    const { status } = req.body;
    const { id } = req.params;

    db.get(`SELECT status FROM orders WHERE id = ?`, [id], (err, order) => {
        if (!order) return res.status(404).json({ error: 'Order not found' });

        const currentStatus = order.status;
        let isValidTransition = false;

        if (currentStatus === 'pending' && status === 'confirmed')
            isValidTransition = true;
        if (currentStatus === 'confirmed' && status === 'shipping')
            isValidTransition = true;
        if (currentStatus === 'shipping' && status === 'delivered')
            isValidTransition = true;

        // LỖI: đơn đã hủy lại được phép thành Delivered!
        if (currentStatus === 'canceled' && status === 'delivered')
            isValidTransition = true;

        if (!isValidTransition) {
            return res.status(400).json({
                error: `Không thể chuyển trạng thái từ ${currentStatus} sang ${status}`
            });
        }

        db.run(`UPDATE orders SET status = ? WHERE id = ?`, [status, id],
        function(err) {
            res.json({ message: 'Update status successfully', status });
        });
    });
});

module.exports = { app, db };

```

5. Quy trình thực hiện đề xuất

| Bước | Công việc | Đầu ra cần đạt | Thời gian |
|------|-----------------|---|------------|
| 1 | Khởi tạo dự án | Cài dependency, sao chép app.js | 10 phút |
| 2 | Test data | Hoàn thành generate-data.js và lifecycle DB | 20–25 phút |
| 3 | Integrity tests | Orphan record, unique constraint, data type | 20–25 phút |
| 4 | API integration | Coupon và state transition tests | 30–35 phút |
| 5 | Báo cáo | Chạy toàn bộ suite, tổng hợp Pass/Fail và fix patch | 15–20 phút |

Khởi tạo và chạy test

```
npm init -y
npm install express sqlite3 jest supertest @faker-js/faker --save-dev
npx jest --verbose
```

6. Tiêu chí đánh giá tham khảo

| Tiêu chí | Trọng số | Mô tả |
|----------------------------|-------------|--|
| Test Data Generation | 20% | Dữ liệu biên đầy đủ; setup/teardown đúng; không rò rỉ dữ liệu giữa các lần chạy. |
| Database Integrity Testing | 25% | Phát hiện orphan record, unique constraint violation và data type inconsistency. |
| Integration Testing | 35% | Đủ case, assertion chính xác; phát hiện sai công thức phần trăm và sai state transition. |
| REPORT.md | 20% | Trình bày rõ; đúng root cause; fix patch hợp lý và khả thi. |
| Tổng cộng | 100% | |

7. Sản phẩm cần nộp

Nén bài dưới dạng .zip, đặt tên thư mục theo mã sinh viên:

Cấu trúc thư mục nộp bài

```
STUDENT_ID_MINILAB_DBTEST/  
├─ generate-data.js  # Sinh dữ liệu test và helper setup DB  
├─ db-logic.test.js  # Toàn bộ test case Jest / Supertest  
└─ REPORT.md         # Báo cáo lỗi và hướng giải quyết
```

Checklist trước khi nộp

- Chạy được `npx jest --verbose` từ thư mục gốc.
- Test độc lập và có thể chạy lặp lại.
- Các test fail có chủ đích được giải thích trong `REPORT.md`.
- Báo cáo nêu rõ bằng chứng, root cause, severity và code fix patch.
- File nén đúng cấu trúc và quy tắc đặt tên.

— Hết đề —

Mục tiêu của bài lab không phải làm mọi test “xanh”, mà là dùng test để chứng minh lỗi tồn tại.